AI Assisted Coding

Week6 -Monday

Name: Lalu prasad Aroori

Roll No.: 2303A51948

Batch No.: 07

Assignment Number: 12.1

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

- **Task 1 – Merge Sort Implementation**

---

1) Prompt to Give in VS Code

Generate a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm.

Requirements:
- Use recursion.
- Include a helper merge function.
- Add a professional docstring.
- Mention time complexity and space complexity in the docstring.
- Include example usage at the bottom.

---

2) Code

```
def merge_sort(arr):
    """
    Sorts a list in ascending order using Merge Sort algorithm.

    Parameters:
        arr (list): List of comparable elements.

    Returns:
        list: Sorted list.

    Time Complexity:
        Best Case: O(n log n)
        Average Case: O(n log n)
        Worst Case: O(n log n)
```

```
    Space Complexity:
        O(n) due to auxiliary arrays used during merging.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)


def merge(left, right):
    merged = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged


# Example usage
arr = [38, 27, 43, 3, 9, 82, 10]
sorted_arr = merge_sort(arr)
print("Sorted array:", sorted_arr)
```

---

3) Output

Sorted array: [3, 9, 10, 27, 38, 43, 82]

---

4) Remarks

✓ Stable sorting algorithm
✓ Guaranteed O(n log n) performance
✓ Suitable for large datasets
❌ Requires extra memory

Merge Sort is ideal when predictable performance is required.

🔹 **Task 2 – Binary Search with AI Optimization**

---

1) Prompt

Generate a Python function binary_search(arr, target) that:
- Searches a sorted list
- Returns index of target
- Returns -1 if not found
- Includes best, average, worst-case time complexity in docstring
- Add comments explaining the logic

---

2) Code

```
def binary_search(arr, target):
    """
    Perform Binary Search on a sorted list.

    Parameters:
        arr (list): Sorted list of elements.
        target: Element to search.

    Returns:
        int: Index of target if found, else -1.

    Time Complexity:
        Best Case: O(1)
        Average Case: O(log n)
        Worst Case: O(log n)

    Space Complexity:
        O(1)
    """
    low = 0
    high = len(arr) - 1
```

```
    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1


# Example
arr = [1, 3, 5, 7, 9, 11]
print(binary_search(arr, 7))
print(binary_search(arr, 4))
```

---

3) Output

```
3
-1
```

---

4) Remarks

✓ Efficient for large sorted datasets
✓ O(log n) performance
❌ Requires sorted input

Binary Search is much faster than Linear Search for large lists.

🔹 Task 3 – Inventory Management System

---

1) Prompt

For an inventory system with thousands of products:
- Recommend efficient searching and sorting algorithms.
- Implement search by product ID and name.
- Implement sorting by price and stock quantity.
- Justify the choice based on dataset size.

---

2) Algorithm Recommendation Table

Operation	Algorithm	Justification
Search by ID	Hash Map (Dictionary)	O(1) lookup
Search by Name	Linear Search / Dictionary	Names may not be unique
Sort by Price	Merge Sort / Timsort	Stable and efficient
Sort by Quantity	Merge Sort	O(n log n)

---

3) Code

```
inventory = [
    {"id": 101, "name": "Laptop", "price": 50000, "stock": 10},
    {"id": 102, "name": "Mouse", "price": 500, "stock": 200},
    {"id": 103, "name": "Keyboard", "price": 1500, "stock": 50},
]

# Search by ID using dictionary
inventory_dict = {item["id"]: item for item in inventory}

def search_by_id(product_id):
    return inventory_dict.get(product_id, "Not Found")

def search_by_name(name):
    return [item for item in inventory if item["name"] == name]

def sort_by_price():
    return sorted(inventory, key=lambda x: x["price"])

def sort_by_stock():
    return sorted(inventory, key=lambda x: x["stock"])
```

---

4) Output

```
search_by_id(101)
→ {'id': 101, 'name': 'Laptop', 'price': 50000, 'stock': 10}
```

---

5) Remarks

✓ Dictionary gives O(1) search
✓ Python sorted() uses Timsort (O(n log n))
✓ Suitable for dynamic updates

🔹 Task 4 – Smart Hospital Patient Management

Recommended Algorithms

Operation	Algorithm	Reason
Search by ID	Hash Map	Fast lookup
Search by Name	Linear Search	Flexible
Sort by Severity	Merge Sort	Stable
Sort by Bill	Merge Sort	Efficient

Code

```
patients = [
    {"id": 1, "name": "Ravi", "severity": 5, "bill": 20000},
    {"id": 2, "name": "Anu", "severity": 9, "bill": 50000},
    {"id": 3, "name": "Kiran", "severity": 3, "bill": 15000},
]

patient_dict = {p["id"]: p for p in patients}

def search_patient(id):
    return patient_dict.get(id, "Not Found")

def sort_by_severity():
    return sorted(patients, key=lambda x: x["severity"], reverse=True)

def sort_by_bill():
    return sorted(patients, key=lambda x: x["bill"])
```

Remarks

✓ Fast emergency prioritization
✓ Efficient billing analysis
✓ Scalable for thousands of records

🔹 Task 5 – University Result Processing

Recommended Algorithms

Operation	Algorithm	Reason
Search by Roll No	Hash Map	O(1)
Sort by Marks	Merge Sort	Stable ranking

Code

```
students = [
    {"roll": 1, "name": "A", "marks": 85},
    {"roll": 2, "name": "B", "marks": 92},
    {"roll": 3, "name": "C", "marks": 75},
]

student_dict = {s["roll"]: s for s in students}

def search_by_roll(roll):
    return student_dict.get(roll, "Not Found")

def sort_by_marks():
    return sorted(students, key=lambda x: x["marks"], reverse=True)
```

🔹 Task 6 – Online Food Delivery Platform

Recommended Algorithms

Operation	Algorithm	Justification
Search by Order ID	Hash Map	Instant lookup
Sort by Delivery Time	Merge Sort	Stable

Operation	Algorithm	Justification
Sort by Price	Merge Sort	Efficient

Code

```
orders = [
    {"id": 1001, "restaurant": "ABC", "time": 30, "price": 250},
    {"id": 1002, "restaurant": "XYZ", "time": 20, "price": 400},
]

order_dict = {o["id"]: o for o in orders}

def search_order(order_id):
    return order_dict.get(order_id, "Not Found")

def sort_by_time():
    return sorted(orders, key=lambda x: x["time"])

def sort_by_price():
    return sorted(orders, key=lambda x: x["price"])
```

✅ **Final Overall Remarks for Lab 12**

✓ Generate optimized algorithms quickly
✓ Add complexity documentation
✓ Suggest better data structures
✓ Improve code readability

❌ Must verify correctness
❌ Must analyze real dataset requirements